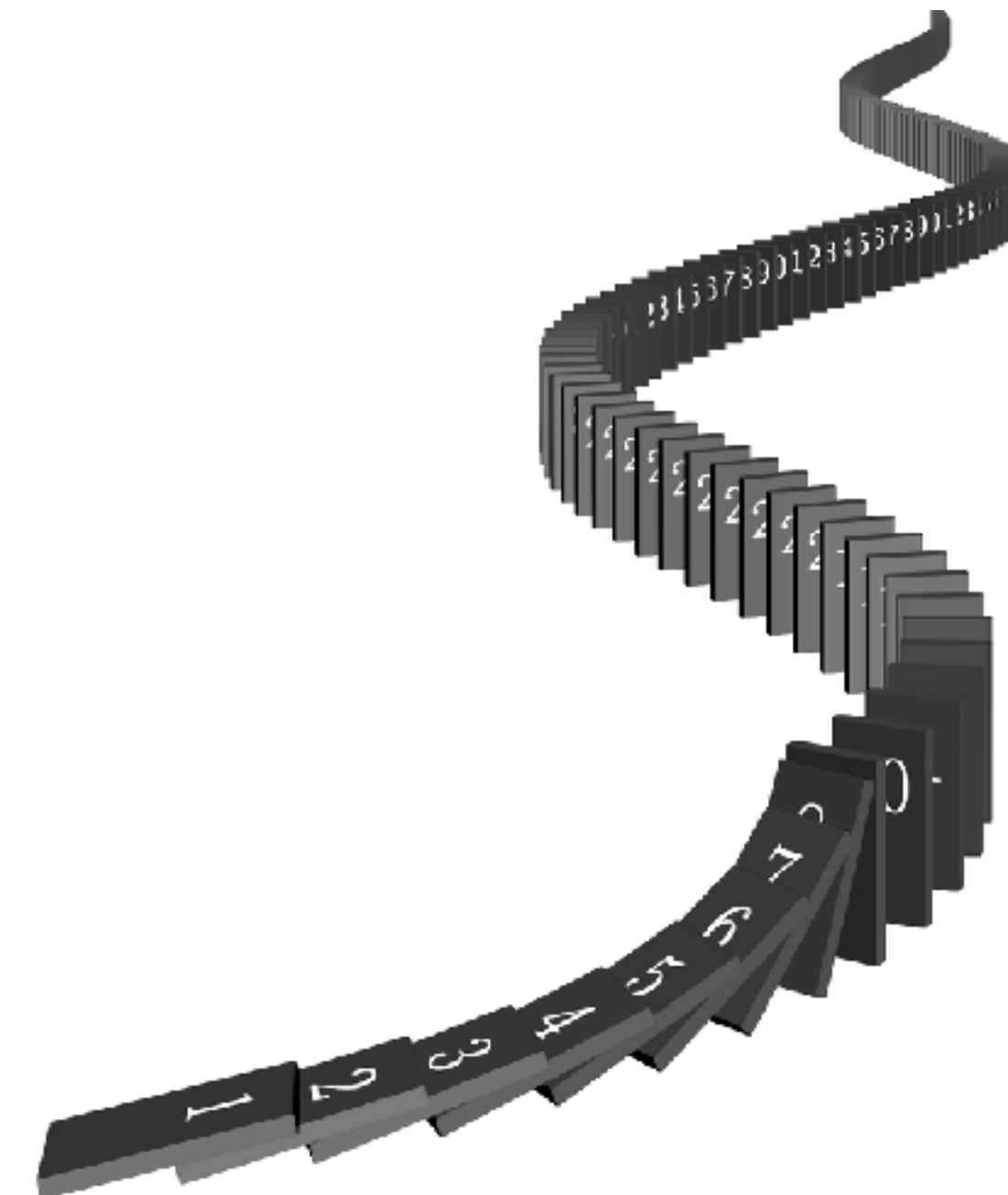


Inducción Matemática en el Análisis de Algoritmos

```
19 template<typename T> unsigned int levenshtein(const T& s1, const T& s2) {
20     const size_t len1 = s1.size(), len2 = s2.size();
21     vector<unsigned int> col(len2+1), prevCol(len2+1);
22     for (unsigned int i = 0; i < prevCol.size(); i++)
23         prevCol[i] = i;
24     for (unsigned int i = 0; i < len1; i++) {
25         col[0] = i+1;
26         for (unsigned int j = 0; j < len2; j++)
27             col[j+1] = std::min( std::min(prevCol[j+1], prevCol[j] + 1, col[j] + 1),
28                                 prevCol[j] + (s1[i] == s2[j] ? 0 : 1) );
29         col.swap(prevCol);
30     }
31     return prevCol[len2];
32 }
33
```



Inducción Matemática

Typical structure of a proof by induction.

Given statement to prove: $\forall n \in \mathbb{N}, P(n)$.

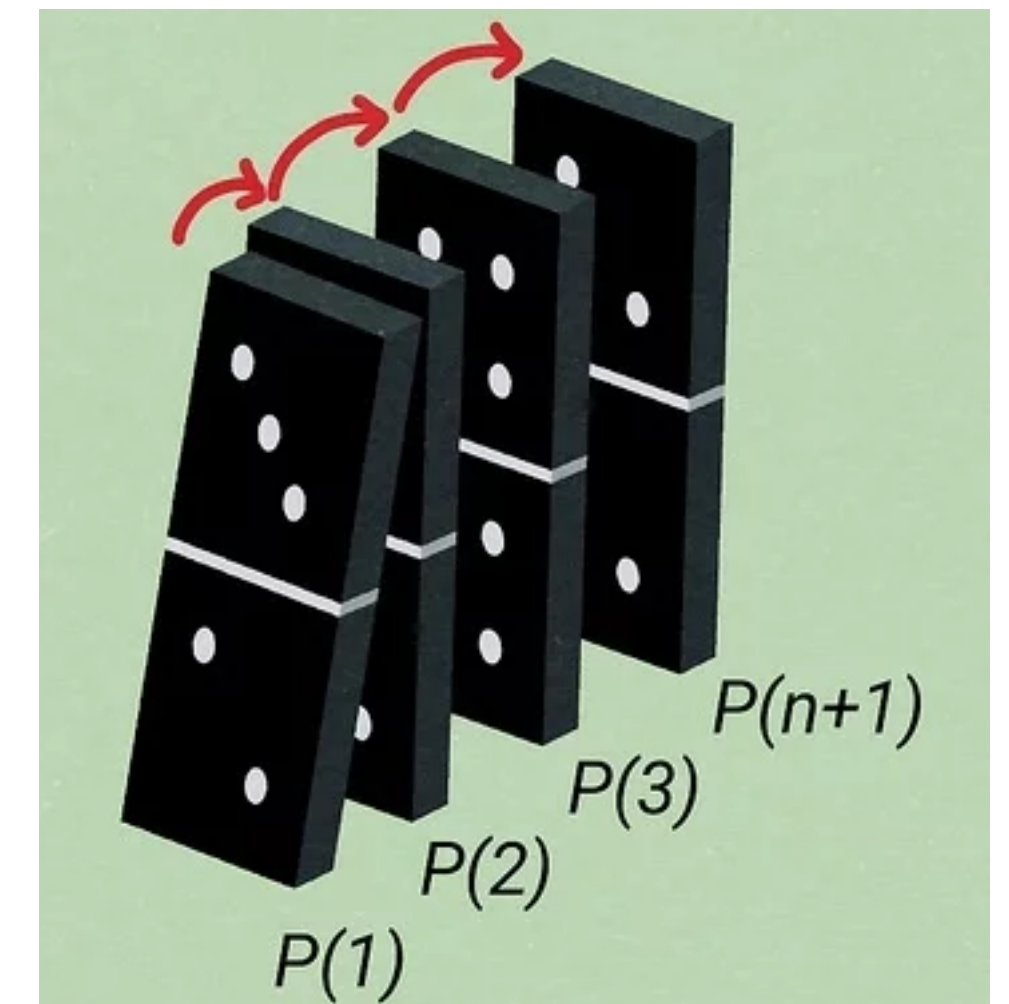
Proof. We prove this by induction on n .

Base case: Let $n = 0$.

[Proof that $P(0)$ is True.]

Inductive step: Let $k \in \mathbb{N}$, and *assume* that $P(k)$ is true. (The assumption that $P(k)$ is true is called the *induction hypothesis*.)

[Proof that $P(k + 1)$ is True.]



Example. Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be defined as $f(n) = \sum_{i=0}^n i$. Prove that for all $n \in \mathbb{N}$,
$$f(n) = \frac{n(n+1)}{2}.$$

Proof. We prove this statement by induction on n .

Base case: Let $n = 0$.

In this case, $f(0) = \sum_{i=0}^0 i = 0$, and $\frac{0(0+1)}{2} = 0$. So the two sides of the equation are equal.

Inductive step: Let $k \in \mathbb{N}$ and assume that $f(k) = \frac{k(k+1)}{2}$.⁴ We want to prove that
$$f(k+1) = \frac{(k+1)(k+2)}{2}.$$

Example. Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be defined as $f(n) = \sum_{i=0}^n i$. Prove that for all $n \in \mathbb{N}$,

$$f(n) = \frac{n(n+1)}{2}.$$

$$\begin{aligned} f(k+1) &= \sum_{i=0}^{k+1} i && \text{(definition of } f\text{)} \\ &= \left(\sum_{i=0}^k i \right) + (k+1) && \text{(taking out last term)} \\ &= f(k) + (k+1) && \text{(definition of } f\text{)} \\ &= \frac{k(k+1)}{2} + (k+1) && \text{(by the I.H.)} \\ &= \frac{(k+1)(k+2)}{2} && \blacksquare \end{aligned}$$

① Prove that $2^n > n, \forall n \in \mathbb{N}$.

When $n = 1$ $2^1 = 2 > 1, \therefore$ true for $n = 1$.

Assume true for $n = k \Rightarrow 2^k > k$

Consider when $n = k + 1$ $2^{k+1} = 2 \cdot 2^k \therefore 2^{k+1} > 2^k$
 $> k + 1 \therefore$ Statement is true for $k + 1$.

As statement is true for $n = 1$ & true for $k \Rightarrow$ true for $k + 1$, $\therefore$ by induction is true
 $\forall n \geq 1, n \in \mathbb{N}$.

① Prove that $2^n > n$, $\forall n \in \mathbb{N}$.

When $n = 1$ $2^1 = 2$ $2 > 1$, $\therefore$ true for $n = 1$.

Assume true for $n = k \Rightarrow 2^k > k$

Consider when $n = k + 1$

$$2^{k+1} = 2 \cdot 2^k \quad \therefore 2^{k+1} > 2^k > k + 1$$

$\therefore$ Statement is true for $k + 1$.

As statement is true for $n = 1$ & true for $k \Rightarrow$ true for $k + 1$, $\therefore$ by induction is true $\forall n \geq 1, n \in \mathbb{N}$.

$$2(2^k) > 2k \geq k+1$$

Consideraciones:

- No necesariamente tiene que empezar en 0 (caso base, primera pieza del dominó).
- Puede restringirse hasta un n finito (número finito de piezas del dominó).
- Idea principal: una pieza tira la siguiente.

Invariantes

Afirmaciones lógica-matemáticas que se cumplen en puntos específicos de la ejecución de un algoritmo.

- Usualmente describen que el algoritmo está tomando buenas “decisiones”.
- Uso típico: ciclos.
- **Objetivo:** probar terminación y corrección; además ayudan a desarrollar un entendimiento real de los algoritmos.
- **No hay una receta** para definir “buenas” invariantes, uno aprende de ver ejemplos.
- Usualmente se demuestran por inducción.

Primer Ejemplo

GALE–SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add h – s to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace h' – s with h – s in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que M es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Demostración. Por inducción en el número x de veces que se ejecuta el inicio del ciclo.

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Demostración. Por inducción en el número x de veces que se ejecuta el inicio del ciclo.

Caso base. Primera vez que se ejecuta **el inicio del ciclo** ($x = 1$). $\mathcal{M}$ es vacío después de la inicialización, por lo tanto es un emparejamiento en ese momento (el ciclo aún no se ha ejecutado).

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Demostración. Por inducción en el número x de veces que se ejecuta el inicio del ciclo.

Caso base. Primera vez que se ejecuta **el inicio del ciclo** ($x = 1$). $\mathcal{M}$ es vacío después de la inicialización, por lo tanto es un emparejamiento en ese momento (el ciclo aún no se ha ejecutado).

Hipótesis de inducción. La afirmación se cumple siempre que el inicio del ciclo se ejecuta k veces ($x = k$). **Ojo: La k -ésima ejecución del ciclo aún no sucede.**

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

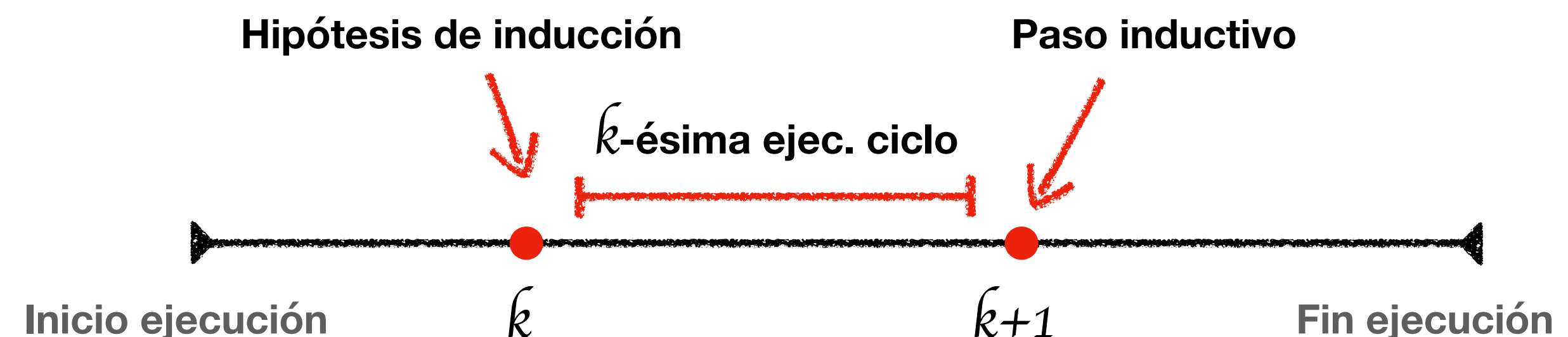
s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Paso inductivo. Vemos que pasa cuando el **inicio del ciclo** se ejecuta por vez $k+1$.

- La hipótesis de inducción implica que $\mathcal{M}$ es un emparejamiento para k , o sea al inicio del ciclo anterior.
- Lo que tenemos que analizar es que pasa durante la k -ésima vez que se ejecuta el cuerpo del ciclo.



Primer Ejemplo

GALE–SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Prueba por casos ($\mathcal{M}$ es el emparejamiento al inicio del ciclo):

- ni h ni s aparecen en $\mathcal{M}$. Entonces $\mathcal{M} \cup \{(h,s)\}$ es un emparejamiento.
- h no aparece en $\mathcal{M}$ y s aparece con h' . Entonces $\mathcal{M} - \{(h',s)\} \cup \{(h,s)\}$ es un emparejamiento.
- $\mathcal{M}$ no se modifica, por lo tanto es un emparejamiento.

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Prueba por casos ($\mathcal{M}$ es el emparejamiento al inicio del ciclo):

- ni h ni s aparecen en $\mathcal{M}$. Entonces $\mathcal{M} \cup \{(h,s)\}$ es un emparejamiento.
- h no aparece en $\mathcal{M}$ y s aparece con h' . Entonces $\mathcal{M} - \{(h',s)\} \cup \{(h,s)\}$ es un emparejamiento.
- $\mathcal{M}$ no se modifica, por lo tanto es un emparejamiento.

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Prueba por casos ($\mathcal{M}$ es el emparejamiento al inicio del ciclo):

- ni h ni s aparecen en $\mathcal{M}$. Entonces $\mathcal{M} \cup \{(h,s)\}$ es un emparejamiento.

- h no aparece en $\mathcal{M}$ y s aparece con h' . Entonces $\mathcal{M} - \{(h',s)\} \cup \{(h,s)\}$ es un emparejamiento.

- $\mathcal{M}$ no se modifica, por lo tanto es un emparejamiento.

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

 IF (s is unmatched)

 Add $h-s$ to matching M .

 ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

 ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Prueba por casos ($\mathcal{M}$ es el emparejamiento al inicio del ciclo):

- ni h ni s aparecen en $\mathcal{M}$. Entonces $\mathcal{M} \cup \{(h,s)\}$ es un emparejamiento.
- h no aparece en $\mathcal{M}$ y s aparece con h' . Entonces $\mathcal{M} - \{(h',s)\} \cup \{(h,s)\}$ es un emparejamiento.
- $\mathcal{M}$ no se modifica, por lo tanto es un emparejamiento.

Primer Ejemplo

GALE-SHAPLEY (*preference lists for hospitals and students*)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't pro

$s \leftarrow$ first student on h 's list to whom h has not yet

IF (s is unmatched)

 Add $h-s$ to matching M .

ELSE IF (s prefers h to current partner h')

 Replace $h'-s$ with $h-s$ in matching M .

ELSE

s rejects h .

RETURN stable matching M .

Invariante. Al **inicio del ciclo**, se cumple que $\mathcal{M}$ es un emparejamiento, es decir, cada hospital o estudiante tiene a lo más una pareja.

Prueba por casos ($\mathcal{M}$ es el emparejamiento al inicio):

1. Si h y s aparecen en $\mathcal{M}$. Entonces $\mathcal{M} \cup \{(h,s)\}$ es un emparejamiento.

2. Si h aparece en $\mathcal{M}$ y s aparece con h' .

3. Si s aparece en $\mathcal{M}$ y h no aparece. Entonces $\mathcal{M} - \{(h',s)\} \cup \{(h,s)\}$ es un emparejamiento.

4. Si s no aparece en $\mathcal{M}$. Entonces $\mathcal{M} \cup \{(h,s)\}$ se modifica, por lo tanto es un emparejamiento.



Segundo Ejemplo

```
Algoritmo minimo(A[], n)
  min = A[1]
  for c = 2 up to n do
    if A[c] < min then min = A[c]
  end for
  return min
end minimo
```

Invariante. La x -ésima vez que se ejecuta el inicio del ciclo, se cumple que mín tiene el mínimo en el rango $\mathcal{A}[1, \dots, x]$.

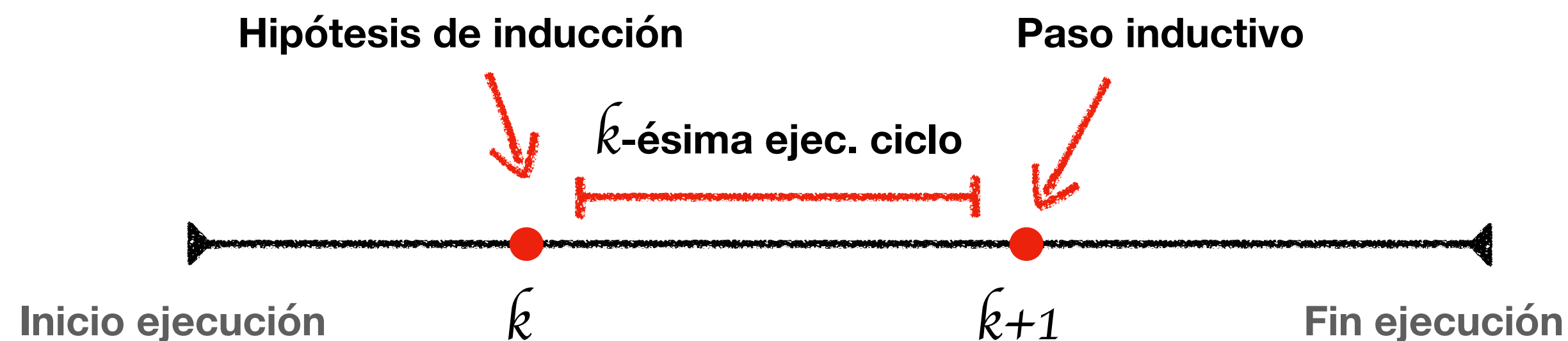
Demostración. Por inducción en x .

Caso Base. Primera vez que se ejecuta el **inicio del ciclo**. Entonces $x = 1$. Por la inicialización, mín es el mínimo en $\mathcal{A}[1, \dots, x] = \mathcal{A}[1]$ (ojo: $c = x+1 = 2$).

Hipótesis Inductiva. La afirmación se cumple cuando el inicio del ciclo se ejecuta k veces ($x = k$).

Segundo Ejemplo

```
Algoritmo minimo(A[], n)
  min = A[1]
  for c = 2 up to n do
    if A[c] < min then min = A[c]
  end for
  return min
end minimo
```



Invariante. La x -ésima vez que se ejecuta el inicio del ciclo, se cumple que mín tiene el mínimo en el rango $\mathcal{A}[1, \dots, x]$.

Paso inductivo. Vemos que pasa cuando el **inicio del ciclo** se ejecuta por vez $k+1$.

- Hipótesis de inducción $\Rightarrow$ invariante es cierta al inicio del ciclo k : mín es el mínimo en $\mathcal{A}[1, \dots, k]$.
- Durante la k -ésima ejecución del ciclo se toma el menor entre mín y $\mathcal{A}[k+1]$ (ojo: $c = k+1$). Por lo tanto, al inicio del ciclo $k+1$, mín tiene el mínimo en $\mathcal{A}[1, \dots, k+1]$.

Segundo Ejemplo

```
Algoritmo minimo(A[], n)
  min = A[1]
  for c = 2 up to n do
    if A[c] < min then
  end for
  return min
end minimo
```



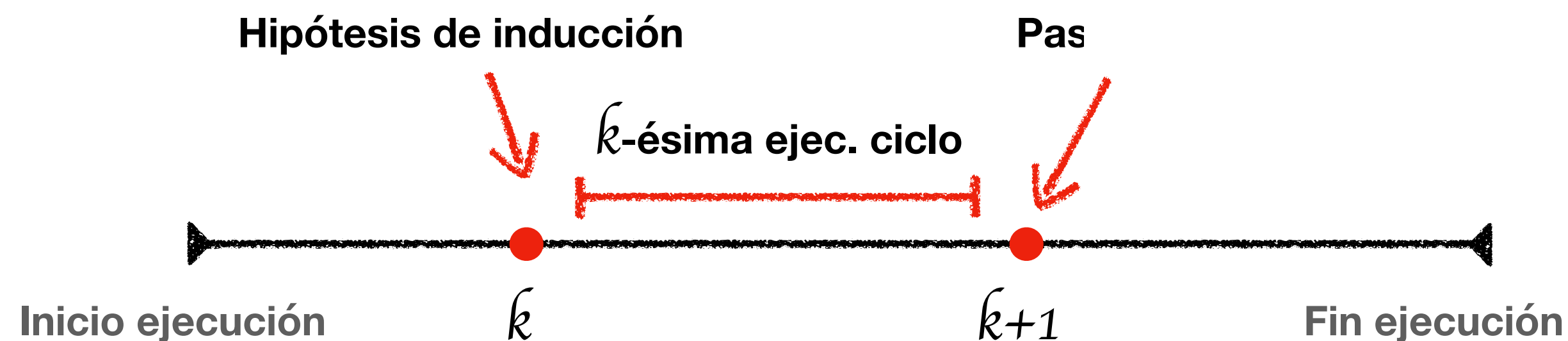
Invariante. La x -ésima vez que se ejecuta el inicio
umple que mín tiene el mínimo en el

.
Vemos que pasa cuando el **inicio del**
por vez $k+1$.

de inducción $\Rightarrow$ invariante es cierta al
ciclo k : mín es el mínimo en $A[1, \dots, k]$.

k -ésima ejecución del ciclo se toma
entre mín y $A[k+1]$ (ojo: $c = k+1$). Por lo

tanto, al inicio del ciclo $k+1$, mín tiene el mínimo
en $A[1, \dots, k+1]$.



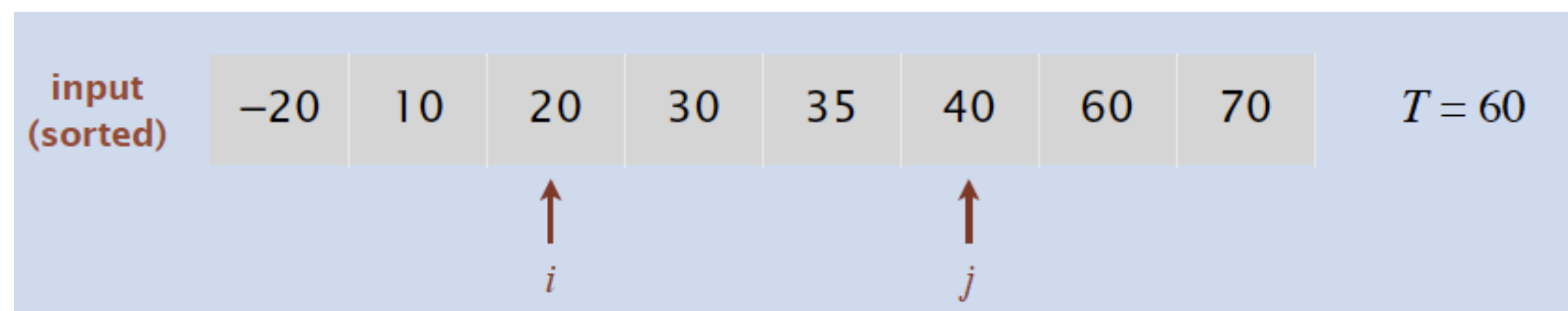
Un Análisis (Casi) Completo

Problema: dado un arreglo $\mathcal{A}$ de enteros ordenados ascendentemente y un entero $\mathcal{T}$, devolver pares de **índices distintos** i, j tales que

$$\mathcal{A}[i] + \mathcal{A}[j] = \mathcal{T},$$

o indicar que no existen tales pares.

```
Algoritmo Target-Sum( $\mathcal{A}[], n, T$ )  
    lo = 1  
    hi = n  
    while lo < hi do  
        sum =  $\mathcal{A}[lo] + \mathcal{A}[hi]$   
        if sum == T then return (lo, hi)  
        else if sum > T then hi = hi - 1  
        else lo = lo + 1  
    end while  
    return NO SE PUEDE :(  
end Target-Sum
```



1a. Invariante

```
Algoritmo Target-Sum(A[],n,T)
    lo = 1
    hi = n
    while lo < hi do
        sum = A[lo] + A[hi]
        if sum == T then return (lo,hi)
        else if sum > T then hi = hi - 1
        else lo = lo + 1
    end while
    return NO SE PUEDE :(
end Target-Sum
```

Invariante 1. La x -ésima vez que se ejecuta el inicio del ciclo, se cumple $hi-lo = n-x$.

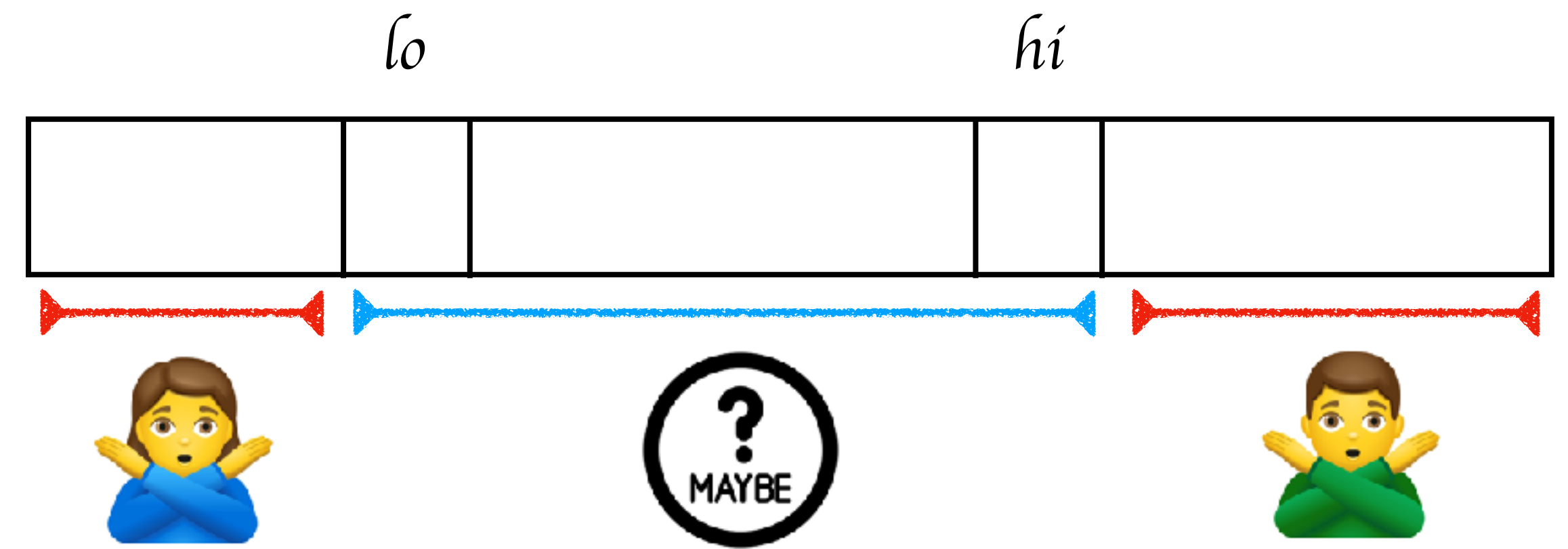
Idea intuitiva: el espacio de búsqueda se **reduce** conforme avanza el cómputo.

=> El algoritmo no se puede ciclar porque para $x = n$, la invariante implica $hi-lo = n-x = 0$, y entonces ya no se ejecuta el ciclo.

2a. Invariante

```
Algoritmo Target-Sum(A[], n, T)
  lo = 1
  hi = n
  while lo < hi do
    sum = A[lo] + A[hi]
    if sum == T then return (lo, hi)
    else if sum > T then hi = hi - 1
    else lo = lo + 1
  end while
  return NO SE PUEDE :(
end Target-Sum
```

Invariante 2. Al inicio del ciclo se cumple que **no existen índices distintos** $i, j \in \{1, \dots, lo-1, hi+1, \dots, n\}$ tales que $A[i] + A[j] = T$.



Idea intuitiva: el espacio de búsqueda se **reduce correctamente** conforme avanza el cómputo.

=> Si devuelve **NO SE PUEDE**, la última vez que se ejecuta el inicio del ciclo se cumple $lo = hi$. Entonces, no hay índices distintos i, j tales que $A[i] + A[j] = T$

Terminación y Corrección

```
Algoritmo Target-Sum(A[],n,T)
  lo = 1
  hi = n
  while lo < hi do
    sum = A[lo] + A[hi]
    if sum == T then return (lo,hi)
    else if sum > T then hi = hi - 1
    else lo = lo + 1
  end while
  return NO SE PUEDE :(
end Target-Sum
```

Terminación:

- **Invariante 1.**

Corrección:

- Si el algoritmo devuelve un par (lo, hi) , entonces $lo \neq hi$ y $A[lo] + A[hi] = T$.
- **Invariante 2** => Si el algoritmo devuelve **NO SE PUEDE**, entonces no hay pares distintos en A que sumen T .

Prueba 2a. Invariante

```
Algoritmo Target-Sum(A[], n, T)
    lo = 1
    hi = n
    while lo < hi do
        sum = A[lo] + A[hi]
        if sum == T then return (lo, hi)
        else if sum > T then hi = hi - 1
        else lo = lo + 1
    end while
    return NO SE PUEDE :(
end Target-Sum
```

Invariante 2. Al inicio del ciclo se cumple que **no existen índices distintos** $i, j \in \{1, \dots, lo-1, hi+1, \dots, n\}$ tales que $A[i] + A[j] = T$.

Demostración. Por inducción en el número x de veces que se ejecuta el inicio del ciclo.

Caso Base. Primera vez que se ejecuta el **inicio del ciclo**. Entonces $x = 1$. Por la inicialización, el conjunto

$$\{1, \dots, lo-1 = 0, hi+1 = n+1, \dots, n\}$$

es vacío. Por lo tanto, la invariante se cumple por **vacuía**.

Hipótesis Inductiva. La afirmación se cumple cuando el inicio del ciclo se ejecuta k veces ($x = k$).

Prueba 2a. Invariante

Algoritmo Target-Sum($A[]$, n , T)

$lo = 1$

$hi = n$

 while $lo < hi$ do

$sum = A[lo] + A[hi]$

 if $sum == T$ then return (lo, hi)

 else if $sum > T$ then $hi = hi - 1$

 else $lo = lo + 1$

 end while

 return NO SE PUEDE :(

end Target-Sum

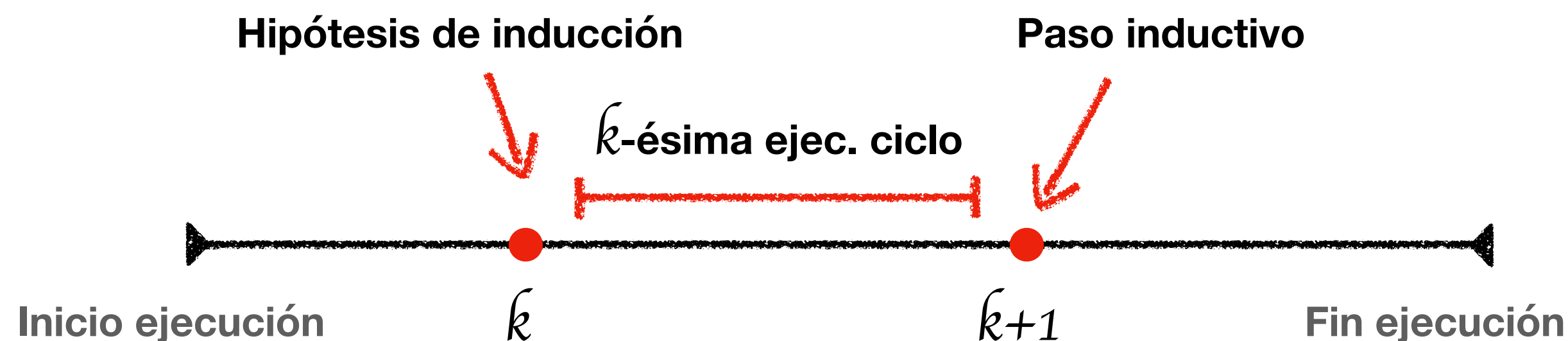
Invariante 2. Al inicio del ciclo se cumple que **no existen índices distintos** $i, j \in \{1, \dots, lo-1, hi+1, \dots, n\}$ tales que $A[i] + A[j] = T$.

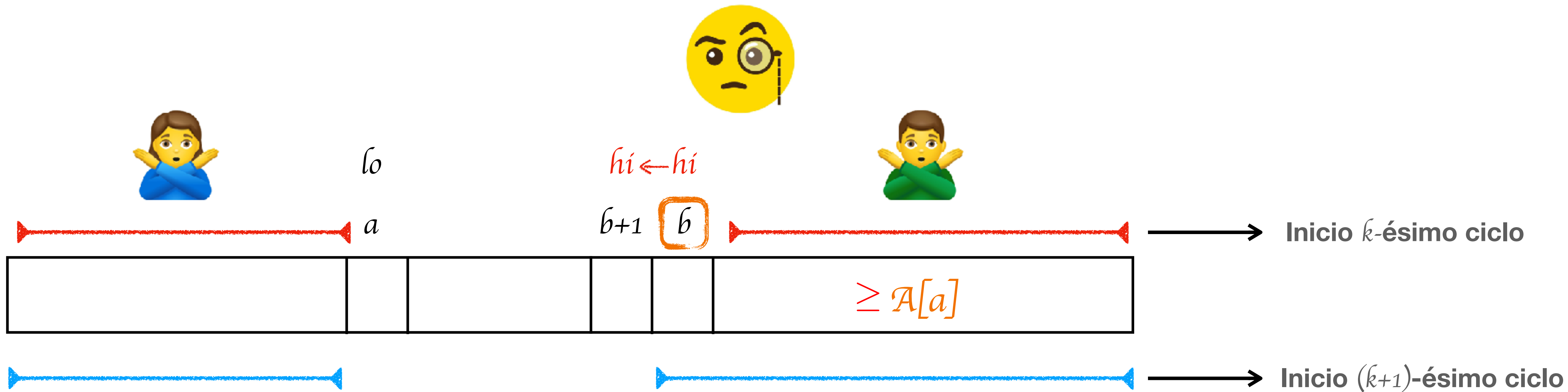
Paso inductivo. Vemos que pasa cuando el **inicio del ciclo** se ejecuta por vez $k+1$.

Hay dos casos a analizar:

- $A[lo] + A[hi] > T$. Entonces hi se “mueve” hacia atrás.
- $A[lo] + A[hi] < T$. Entonces lo se “mueve” hacia adelante.

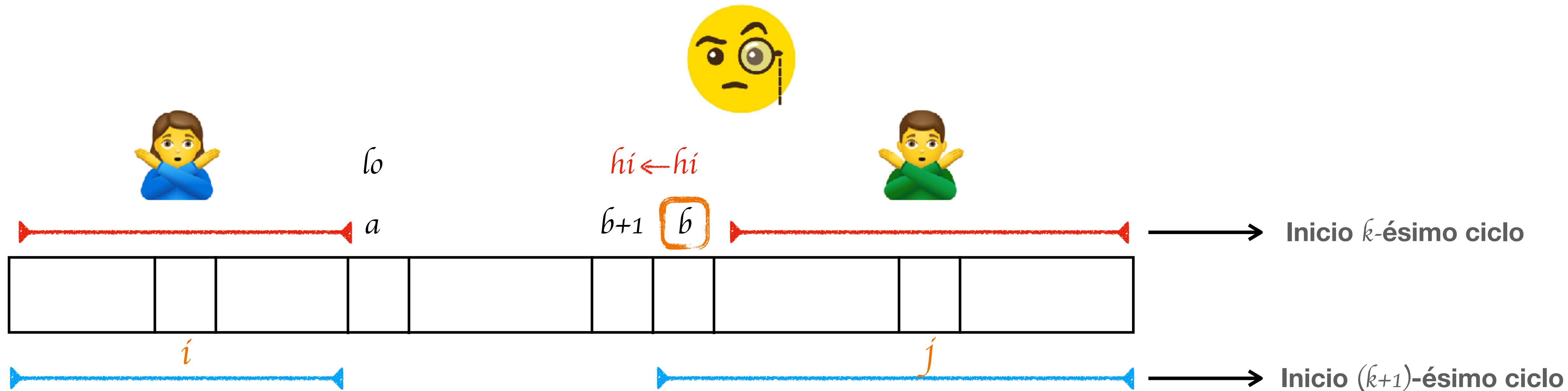
Análisis **simétrico**. Veremos solo el primero.





Caso 1. $\forall j \in \{b+1, \dots, n\}, \mathcal{A}[b] + \mathcal{A}[j] \neq \mathcal{T}$.

- hi se mueve hacia atrás en la iteración $k \Rightarrow \mathcal{A}[a] + \mathcal{A}[b] > \mathcal{T}$.
- $\mathcal{A}$ está ordenado $\Rightarrow \mathcal{A}[j] \geq \mathcal{A}[a]$.
- $\Rightarrow \mathcal{A}[b] + \mathcal{A}[j] > \mathcal{T}$. Lo que buscábamos probar.



Caso 2. $\forall i \in \{1, \dots, a-1\}, A[i] + A[b] \neq \mathcal{T}$.

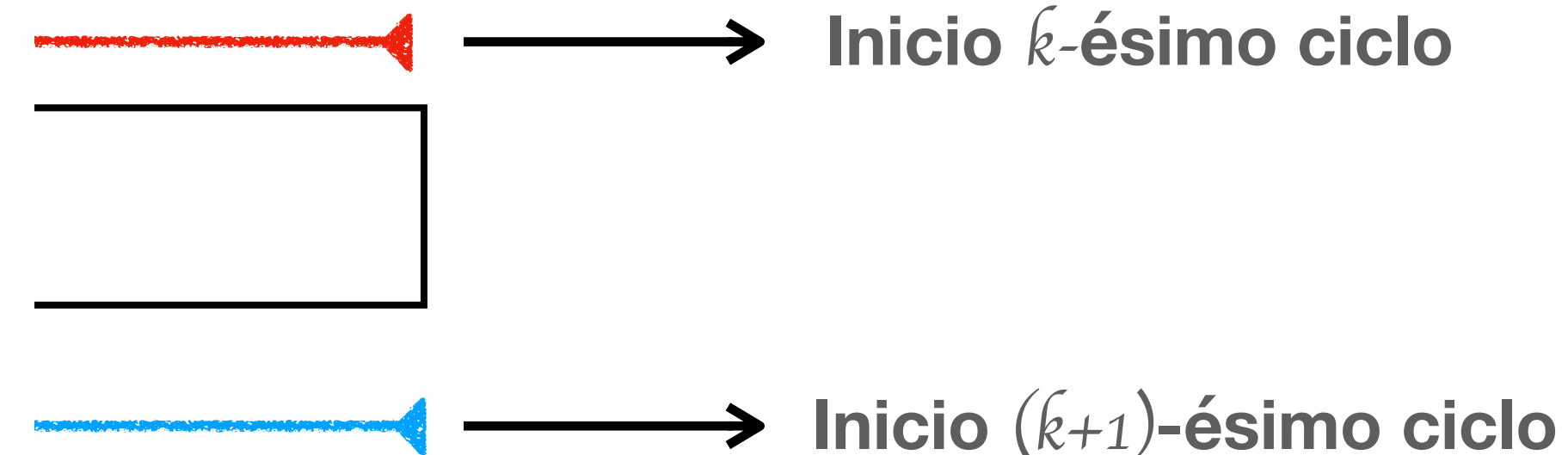
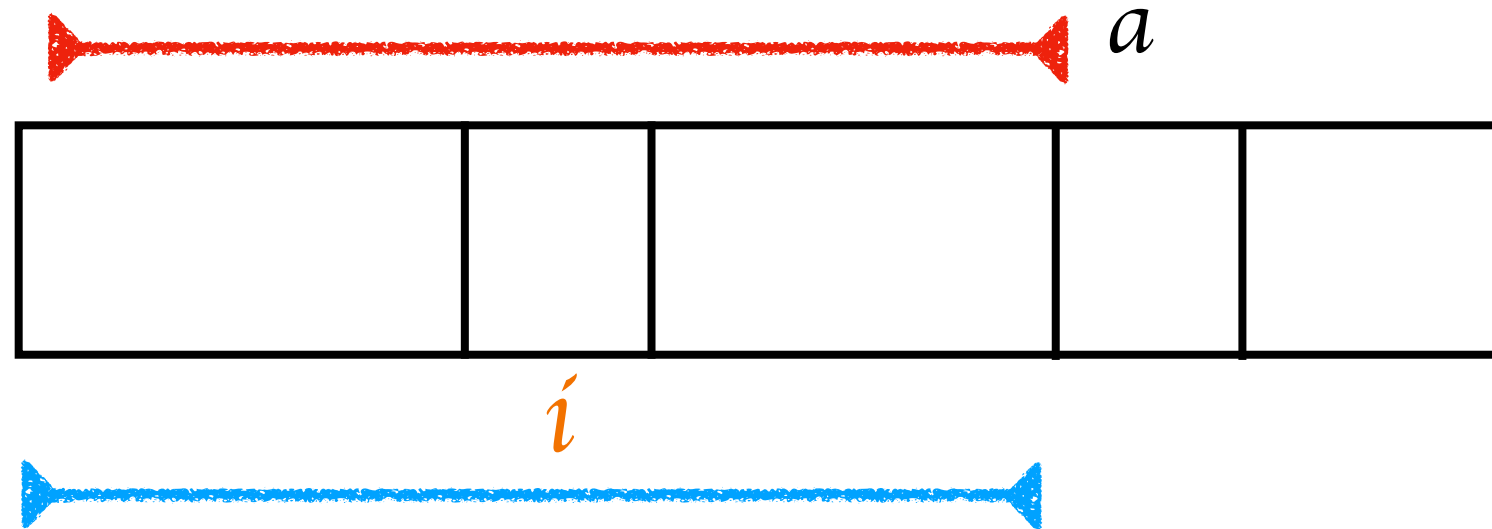
- Por contradicción, supongamos que $\exists i \in \{1, \dots, a-1\}, A[i] + A[b] = \mathcal{T}$
- l_0 solo avanza y h_i solo retrocede $\Rightarrow$ En algún ciclo m anterior al k -ésimo, $l_0 = i$ y l_0 avanzó en ese ciclo; en el m -ésimo ciclo $h_i = j$, con $j \in \{b+1, \dots, n\}$.
- Como h_i avanzó en el ciclo m -ésimo $\Rightarrow A[i] + A[j] < \mathcal{T}$.
- $A[b] \leq A[j]$ (pues A está ordenado) $\Rightarrow A[i] + A[b] < \mathcal{T}$. Contradicción!



l_0



$h_i \leftarrow h_i$



Caso 2. $\forall i \in \{1, \dots, a-1\}, A[i]$

- Por contradicción, sup
- l_0 solo avanza y h_i s

l_0 avanzó en ese ciclo; en el m -ésimo ciclo $m = j$, con $j \in \{b+1, \dots, n\}$.

- Como h_i avanzó en el ciclo m -ésimo $\Rightarrow A[i] + A[j] < T$.
- $A[b] \leq A[j]$ (pues A está ordenado) $\Rightarrow A[i] + A[b] < T$. Contradicción!

$$A[b] = T$$

m anterior al k -ésimo, $l_0 = i$ y

Tiempo de ejecución

```
Algoritmo Target-Sum(A[], n, T)
  lo = 1
  hi = n
  while lo < hi do
    sum = A[lo] + A[hi]
    if sum == T then return (lo, hi)
    else if sum > T then hi = hi - 1
    else lo = lo + 1
  end while
  return NO SE PUEDE : (
end Target-Sum
```

Annotations in the original image:

- A red vertical bracket next to `lo = 1` and `hi = n` is labeled C_1 .
- A red arrow points from `while lo < hi do` to $n * C_2$.
- A red vertical bracket next to the `if`, `else if`, and `else` branches is labeled $(n-1) * C_3$.
- A red arrow points from `return NO SE PUEDE :` to 1 .

$$T(n) = (C_2 + C_3)n + C_1 - C_3 + 1$$

Tiempo de ejecución

$$T'(n) = (C_2 + C_3)n + C_1 - C_3 + 1 = O(n)$$

- ♦ Observemos que $(C_1 + C_2 + 1)n = (C_1 + C_2 + C_3 - C_3 + 1)n = (C_2 + C_3)n + (C_1 - C_3 + 1)n$.
- ♦ Por lo tanto, para todo $n \geq 1$,
$$T'(n) = (C_2 + C_3)n + (C_1 - C_3 + 1) \leq (C_2 + C_3)n + (C_1 - C_3 + 1)n = (C_1 + C_2 + 1)n.$$
- ♦ Entonces, $C = C_1 + C_2 + 1$, $n_0 = 1$ son constantes que prueban $T'(n) = O(n)$.

Upper bounds. $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.